

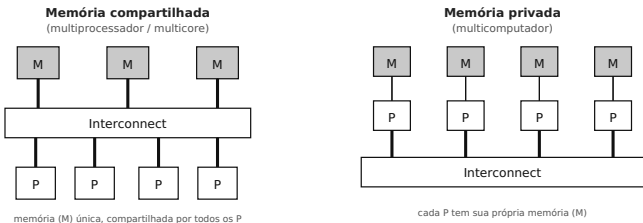
Aula 02 — Introdução (parte 2)

Resumo 3 de 14 · arquivo aula02 · 20 slides · Cap. 1 de van Steen & Tanenbaum

Esta aula apresenta uma **classificação** dos sistemas distribuídos em três grandes grupos — (1) **computação de alto desempenho**, (2) **sistemas de informação distribuídos** e (3) **sistemas pervasivos** — e termina com as **armadilhas** (falsas suposições) que tornam esses sistemas desnecessariamente complexos. É uma aula de vocabulário: muitos termos que reaparecerão no curso.

1. Computação distribuída de alto desempenho

A computação distribuída de alto desempenho começou com a **computação paralela**. A distinção fundamental de hardware é entre **multiprocessador/multicore** (memória compartilhada) e **multicomputador** (memória privada):



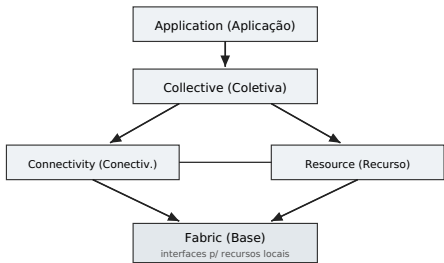
Multiprocessador (memória compartilhada) × multicomputador (memória privada). Multiprocessadores são mais fáceis de programar, mas escalam mal com muitos processadores.

Como os multiprocessadores escalam mal, tentou-se a **memória compartilhada distribuída (DSM)**: simular memória compartilhada sobre um multicomputador, mapeando páginas de memória de vários nós num único espaço virtual (se A acessa uma página em B, o SO busca de B como se buscasse do disco). Na prática o desempenho nunca competiu e a técnica foi **abandonada**.

Os dois principais agrupamentos físicos de alto desempenho:

- **Cluster**: grupo de máquinas ligadas por uma LAN; **homogêneo** (mesmo SO, hardware quase idêntico), com poucos nós de gerenciamento.
- **Grid**: muitos nós em vários lugares; **heterogêneo**, disperso entre **várias organizações**, podendo abranger uma WAN. Usa *organizações virtuais* (agrupamento de IDs de usuários) para controlar acesso/autorização na alocação de recursos.

A **arquitetura de grid** é organizada em cinco camadas:



Camadas do grid: Fabric (base, interfaces locais) → Connectivity (comunicação + autenticação) e Resource (gerencia 1 recurso) → Collective (vários recursos: descoberta, agendamento, replicação) → Application.

2. Sistemas de informação distribuídos

Nascem da necessidade de **integrar** muitas aplicações em rede de uma organização (interoperabilidade sempre foi dolorosa). Uma **aplicação em rede** roda num servidor e oferece serviços a clientes remotos; a integração simples faz o cliente combinar requisições a várias aplicações, enviá-las e montar um resultado coerente. O passo seguinte é a comunicação *direta* entre aplicações — a **Integração de Aplicações Empresariais (EAI)**.

Transações e ACID

Uma **transação** agrupa operações num bloco “tudo-ou-nada”. Primitivas: INICIAR_TRANSACAO, TERMINAR_TRANSACAO (confirma), ABORTAR_TRANSACAO (desfaz), LER, ESCREVER. Toda transação deve ter as propriedades **ACID**:

Propriedade	Significado
Atomicidade	Ocorre de forma indivisível (aparentemente tudo ou nada)
Consistência	Não viola invariantes do sistema
Isolamento	Transações concorrentes não interferem entre si
Durabilidade	Após confirmar, as mudanças são permanentes

Quando os dados estão espalhados por vários servidores, um **TPM (Transaction Processing Monitor)** coordena a execução da transação distribuída.

Middleware de integração

O middleware oferece a “cola” de comunicação. Dois modelos centrais:

- **RPC (Remote Procedure Call):** a requisição parece uma chamada de função local, mas é empacotada como mensagem, enviada, processada, e o resultado volta como retorno da chamada. Exige que chamador e chamado estejam *ativos ao mesmo tempo*.
- **MOM (Middleware Orientado a Mensagens):** mensagens vão a um ponto de contato lógico e são encaminhadas às aplicações interessadas (modelo produtor/consumidor). Permite **desacoplamento no tempo e no espaço**.

Formas de integrar aplicações, do mais rígido ao mais flexível: **transferência de arquivos** (simples, pouco flexível), **banco de dados compartilhado** (flexível, mas exige esquema comum e pode virar gargalo), **RPC** (bom para uma sequência de ações) e **mensagens** (desacopla no tempo/espaço).

3. Sistemas pervasivos distribuídos

Geração de sistemas com nós **pequenos, móveis e embutidos** que se integram naturalmente ao ambiente do usuário. Há três subtipos (que se sobrepõem):

Computação ubíqua

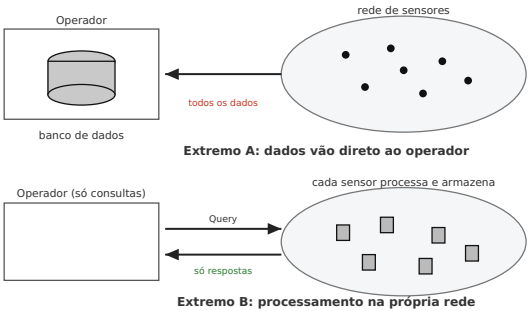
Pervasiva e continuamente presente (interação contínua sistema↔usuário). Cinco elementos: **distribuição** (dispositivos em rede, acessíveis de forma transparente), **interação** discreta, **consciência de contexto** (onde/quem/quando/o quê), **autonomia** (operam sem intervenção, autogerenciados) e **inteligência** (lida com ações dinâmicas variadas).

Computação móvel

Ênfase na **mobilidade**: muitos tipos de dispositivos (smartphones, tablets, GPS); a localização muda com o tempo, então a palavra-chave é **descoberta** de serviços locais; manter comunicação estável é desafiador. O compartilhamento direto de recursos entre dispositivos móveis nunca emplacou. Na prática, dispositivos móveis se conectam a **servidores estacionários**, virando clientes de serviços em nuvem — daí *mobile cloud computing* e *mobile edge computing*.

Redes de sensores

Nós **muitos** (dezenas a milhares), **simples** (pouca memória/CPU/comunicação) e geralmente **a bateria**. Podem ser vistas como bancos de dados distribuídos, com dois extremos de projeto:



Redes de sensores como BD distribuído: (A) todos os dados brutos são armazenados no operador; (B) cada sensor processa/armazena localmente e responde apenas às consultas (menos tráfego, economiza bateria).

4. Armadilhas: as falsas suposições

Muitos sistemas distribuídos ficam desnecessariamente complexos por causa de **suposições falsas e ocultas**. São as clássicas “falácias da computação distribuída” — assumir cada uma delas leva a erros graves:

- A rede é **confiável**
- A rede é **segura**
- A rede é **homogênea**
- A **topologia** da rede não muda
- A **latência** é zero
- A **largura de banda** é infinita
- O **custo de transporte** é zero
- Há um **único administrador**